

Playwright Capstone Project Planner

Project Name: Enterprise Quality Engineering Automation Framework

Target Application: OrangeHRM Open Source Enterprise Demo

Selected Website: <https://opensource-demo.orangehrmlive.com/>

Document Version: 4.0 (Core Playwright & OrangeHRM Edition)

1. Application Context: Why OrangeHRM?

OrangeHRM is a world-class, open-source Human Resource Management (HRM) system utilized by enterprises globally. For this Capstone Project, OrangeHRM serves as the perfect target application because it naturally contains all the complex UI/UX components required to validate an engineer's Playwright capabilities.

- **Data-Heavy Grids:** The Employee (PIM) and Admin lists provide dynamic tables requiring advanced array iteration, sorting validation, and pagination handling.
- **Complex Form Elements:** The Recruitment and User Profile modules utilize customized non-select dropdowns, multi-step wizards, date pickers, and asynchronous file uploads.
- **SVG Graphics Rendering:** The main Dashboard relies on dynamic, coordinate-based SVG pie charts and timesheet graphs, fulfilling the "Project #2" low-level DOM manipulation requirement.
- **Network & API Architecture:** It utilizes a modern backend API, making it ideal for practicing `page.route()` backend mocking (e.g., intercepting JSON responses) and session cookie extraction for API-based logins.

2. Project Objectives

- **Scale:** Develop 120 automated test cases across 8 OrangeHRM modules (15 tests per module).
- **Architecture:** Use a reusable Page Object Model (POM) and data-driven testing.
- **Resiliency:** Write stable, parallel-ready tests with robust locators and no hard-coded waits.
- **Advanced Features:** Handle API mocking, file uploads, visual regression masking, and SVG extraction.
- **Reporting:** Generate advanced HTML results using Allure Report.

3. Technology & Automation Stack

Technology	Function in Framework
Playwright v1.4x	Core engine for interacting with OrangeHRM UI, extracting SVGs, and intercepting APIs.
JavaScript / Node.js	Core programming language and runtime environment.
Git & GitHub	Version control, branching strategies, and repository management.
Faker.js	Generating randomized, unique test data (Employee names, emails, credentials).
dotenv	Managing environment variables securely (Base URLs, SuperAdmin Credentials).
Allure Report	Advanced reporting suite capturing DOM snapshots, videos, and analytics trends.
GitHub Copilot / GenAI	Local AI acceleration for refactoring brittle CSS into semantic getByRole locators.

4. Planned Modules / Services

- **Module 1: Authentication & Security**
Testing login workflows, session management, and bypassing the UI via API for faster test setup.
- **Module 2: Employee Data Grid / PIM** Verifying the employee table functionality, including sorting, pagination, and dynamic row actions.
- **Module 3: Recruitment Forms**
Automating candidate application forms, including complex dropdowns and resume file uploads.

- **Module 4: Dashboard Analytics**
Validating graphical dashboard widgets and extracting data from dynamic SVG charts.
- **Module 5: Leave Management**
Testing the leave application process, handling calendar date-picking, and browser dialog popups.
- **Module 6: "My Info" Profile**
Checking user profile updates, field limits, and profile picture upload/download functionalities.
- **Module 7: Time Tracking**
Performing visual regression testing on timesheets while masking dynamic clock and date values.
- **Module 8: Admin Panel**
Mocking backend API responses to test how the UI handles fake data injections and server errors.

5. Execution Timeline

Day	Planned Work
Day 1	Project setup, folder structure initialization, and configuration.
Day 2	Automate Authentication and Employee Data Grid modules.
Day 3	Automate Recruitment Forms and Dashboard Analytics modules.
Day 4	Automate Leave Management and "My Info" Profile modules.
Day 5	Automate Time Tracking (Visuals) and Admin Panel (API Mocking) modules.
Day 6	Code refactoring, test debugging, and pushing final code to GitHub.
Day 7	Generate Allure reports and prepare the final project documentation.

5. Framework Directory Structure

Playwright-OrangeHRM-Capstone/

```
|— api/                                # API bypass controllers (auth bypass, token
generators)
|— data/                                # Data-driven JSON arrays for form testing
|— pages/                               # Page Object Model classes
|   |— components/                     # Reusable widgets (Navbar, PaginationTable, SVGChart)
|   |— AdminPage.js                    # OrangeHRM Admin view mapping
|   |— PIMPage.js                      # OrangeHRM PIM view mapping
|   └─ RecruitmentPage.js
|— tests/                               # specs strictly grouped by module
|   |— 01_auth.spec.js
|   |— 02_pim.spec.js
|   |— 03_recruitment.spec.js
|   |— 04_dashboard.spec.js
|   |— 05_leave.spec.js
|   |— 06_myinfo.spec.js
|   |— 07_time.spec.js
|   └─ 08_admin_mocking.spec.js
|— utils/                              # Custom hooks, soft-assertions, faker-generators
|— screenshots/                        # Baseline visual regression files
|— playwright-report/                  # Default HTML report output
|— allure-results/                     # Raw Allure data (processed later via 'allure
generate')
|— playwright.config.js                # Workers, retries, reporting, and parallel config
|— .env                               # Environment variables (Ignored in Git)
|— .gitignore
|— package.json
└─ README.md
```

7. Expected Outcomes

By the end of this project, the final repository will demonstrate:

- A fully functional, maintainable automation framework using the Page Object Model (POM).
- Automated test cases executing reliably in parallel without session interference.
- Practical mastery of complex automation challenges like tables, iframes, file uploads, and SVGs.
- Improved test execution speed using API mocking and programmatic login techniques.
- A comprehensive Allure dashboard displaying test trends, screenshots, and failure traces.